EPISODE 3

Node.js, MongoDB & Backend Architecture

Build Production-Ready REST APIs

Node.js & Express	MongoDB & Mongoose
REST API Design	Authentication (JWT)
Middleware & Security	WebSockets & Socket.io
Redis Caching	Testing & Production

TABLE OF CONTENTS

- Passport.js

12 **WebSockets & Socket.io**

- Real-time comms
- Rooms & middleware

13 **Redis Caching**

- Cache patterns
- TTL strategies
- Pub/Sub

14 **Security & Error Handling**

- XSS, CSRF, DoS
- Rate limiting
- Error middleware

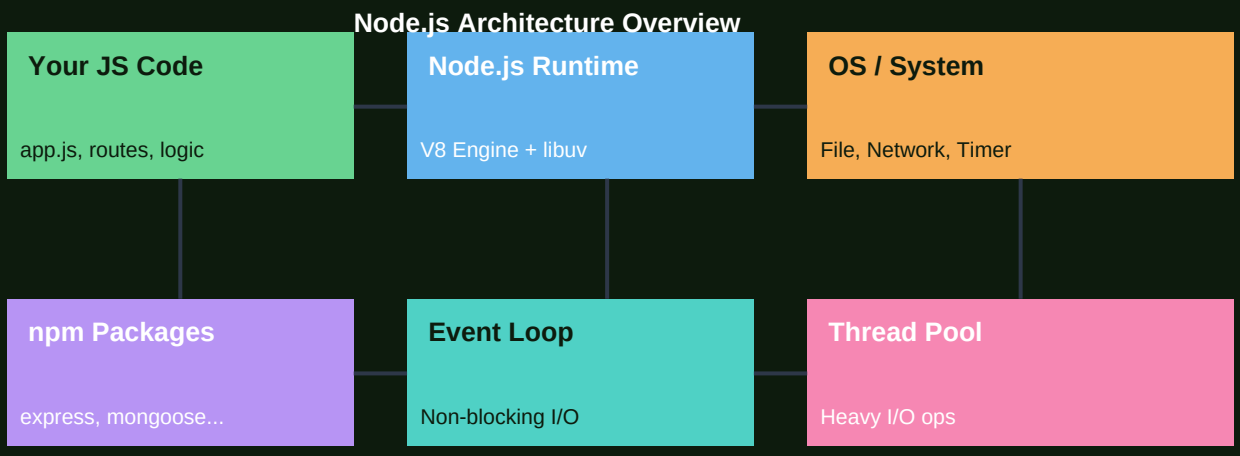
15 **Production & Testing**

- Project structure
- PM2
- Jest unit testing

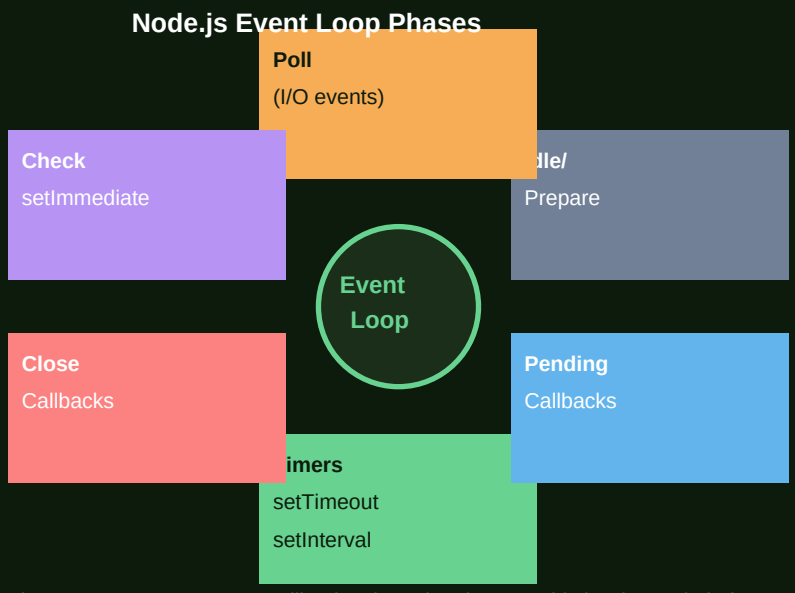
CHAPTER 1 — NODE.JS FOUNDATIONS

What is Node.js?

Node.js is a JavaScript runtime built on Chrome's V8 engine. It lets JavaScript run OUTSIDE the browser — on servers, CLIs, scripts. Its killer feature is the non-blocking, event-driven architecture: instead of waiting for I/O (files, network, DB) to complete, Node registers a callback and moves on to the next task.



Node.js runtime: your code runs in V8, I/O goes through libuv's thread pool



The Event Loop processes callbacks phase by phase — this is why Node is fast

Why Node.js for Backend?

✓ Same language (JS) on frontend and backend — one team, shared code.
✓ Non-blocking I/O: handles thousands of concurrent connections with one thread.
✓ Huge ecosystem: 2M+ npm packages for almost any use case.
✓ Fast for I/O-heavy workloads: APIs, real-time apps, microservices.
✗ Not ideal for CPU-heavy tasks (image processing, ML) — blocks the loop.
✗ Callback-heavy code can become complex — solved by Promises + async/await.

```
// package.json — project manifest
{
  "name": "my-api",
  "version": "1.0.0",
  "scripts": {
    "start": "node src/index.js",
    "dev":   "nodemon src/index.js",
    "test":  "jest"
  },
  "dependencies": {
    "express":  "^4.18.2",
    "mongoose": "^8.0.0",
    "jsonwebtoken": "^9.0.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.0",
    "jest": "^29.0.0"
  }
}
```

CHAPTER 2 — CREATING YOUR FIRST SERVER

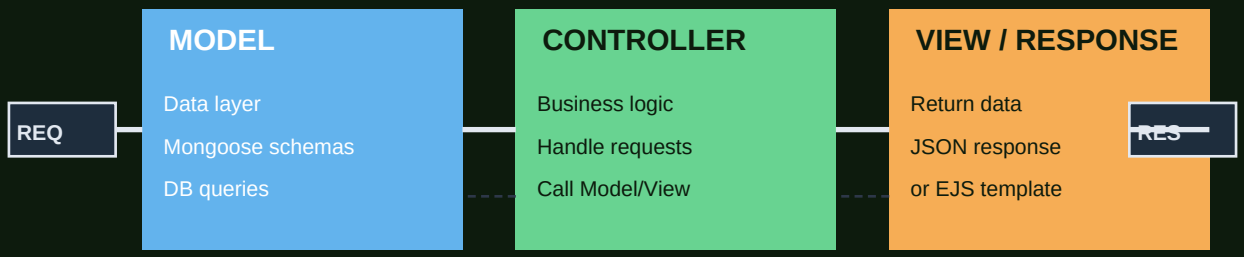
Native HTTP Server → Express

```
// Native Node.js server (before Express)
const http = require('http');

const server = http.createServer((req, res) => {
  const url    = req.url;
  const method = req.method;

  if (url === '/' && method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Namaste Duniya! ■');
  } else if (url === '/api/users' && method === 'GET') {
    res.writeHead(200, { 'Content-Type': 'application/json' });
    res.end(JSON.stringify({ users: [] }));
  } else {
    res.writeHead(404); res.end('Not found');
  }
});

server.listen(3000, () => console.log('Server running on :3000'));
```

Code	Category	Meaning	When Sent
1xx	Informational	Processing in progress	100 Continue, 101 WS Upgrade
2xx	Success	Request succeeded	200 OK, 201 Created, 204 No Content
3xx	Redirect	Resource moved	301 Permanent, 302 Temporary, 304 Cached
4xx	Client Error	Request problem	400 Bad, 401 Auth, 403 Forbidden, 404 Missing, 422 Invalid
5xx	Server Error	Server failed	500 Internal, 502 Bad Gateway, 503 Unavailable

CHAPTER 3 — MVC ARCHITECTURE

Model-View-Controller Pattern



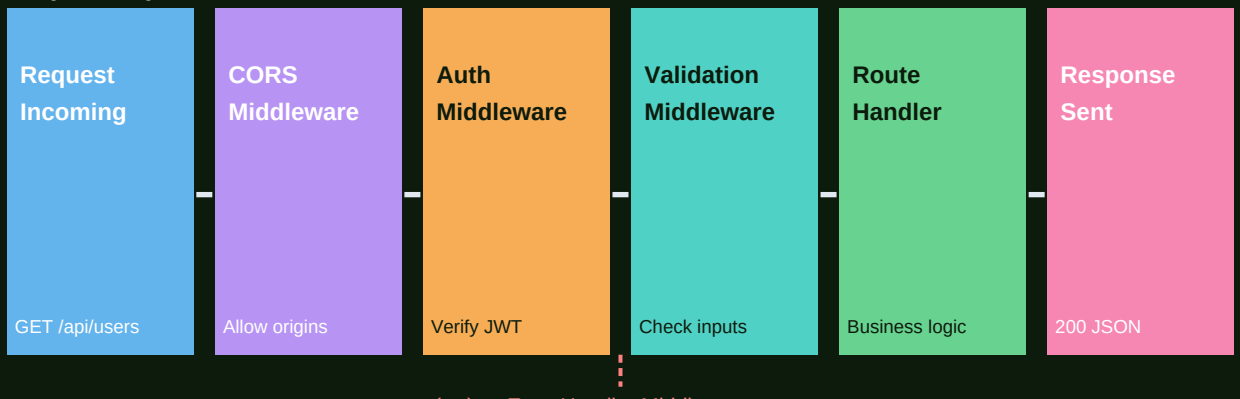
MVC separates concerns: data (Model), logic (Controller), output (View/Response)

```
// Folder structure following MVC
src/
■■■ models/
■   ■■■ user.model.js
■   ■■■ post.model.js
■■■ controllers/
■   ■■■ user.controller.js
■   ■■■ auth.controller.js
■■■ routes/
■   ■■■ user.routes.js
■   ■■■ auth.routes.js
■■■ middleware/
■   ■■■ auth.middleware.js
■   ■■■ validate.middleware.js
■■■ config/
■   ■■■ db.js
■■■ utils/
■   ■■■ ApiError.js
■■■ index.js
```

CHAPTER 4 — EXPRESS.JS FRAMEWORK

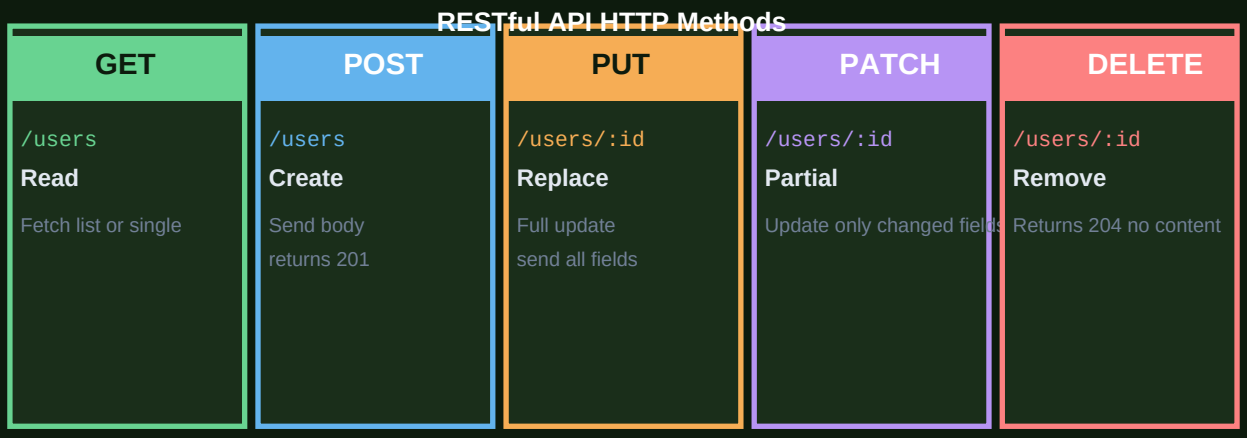
Building with Express

Request Pipeline



Every request passes through middleware functions before reaching your route handler

```
const express = require('express');
const app = express();

// Built-in middleware
app.use(express.json());           // parse JSON body
app.use(express.urlencoded({ extended: true })); // parse form data
app.use(express.static('public')); // serve static files

// Route with URL params: GET /users/42
app.get('/users/:id', async (req, res) => {
  const { id } = req.params;      // URL param
  const { fields } = req.query;   // query string ?fields=name,email
  const user = await User.findById(id);
  if (!user) return res.status(404).json({ error: 'User not found' });
  res.json(user);
});

// POST with body
app.post('/users', async (req, res) => {
  const { name, email, password } = req.body;
  const user = await User.create({ name, email, password });
  res.status(201).json(user);
});

app.listen(3000);
```



RESTful HTTP methods: each verb has a semantic meaning — follow conventions

CHAPTER 5 — TEMPLATE ENGINES (EJS)

Server-Side Rendering with EJS

EJS (Embedded JavaScript) lets you inject dynamic data into HTML templates on the server. While modern apps use React/Next.js, EJS is still used for server-rendered admin panels and emails.

Tag	Purpose	Example
<%= %>	Output escaped value	<%= user.name %>
<%- %>	Output raw HTML (unescaped)	<%- htmlContent %>
<% %>	Execute JS (no output)	<% if (isAdmin) { %>
<%# %>	Comment (not rendered)	<%# This is a comment %>
<%- include() %>	Include partial file	<%- include('partials/nav') %>

```
// Express setup
app.set('view engine', 'ejs');
app.set('views', './views');

// Route passes data to template
app.get('/dashboard', async (req, res) => {
  const users = await User.find().limit(10);
  res.render('dashboard', { users, title: 'Dashboard', isAdmin: true });
});

<!-- views/dashboard.ejs -->
<!DOCTYPE html>
<html>
<head><title><%= title %></title></head>
<body>
  <% if (isAdmin) { %>
    <a href='/admin'>Admin Panel</a>
  <% } %>

  <ul>
    <% users.forEach(user => { %>
      <li><%= user.name %> — <%= user.email %></li>
    <% }) %>
  </ul>
</body>
</html>
```

CHAPTER 6 — MIDDLEWARE DEEP DIVE

How Middleware Works

Middleware is a function that receives (req, res, next). It can modify the request, send a response, or call next() to pass control to the next middleware. This chain is the backbone of every Express application.

```
// Custom middleware — logging
const logger = (req, res, next) => {
  console.log(`${new Date().toISOString()} ${req.method} ${req.url}`);
  next(); // MUST call next() or request hangs!
};
app.use(logger);

// Auth middleware
const requireAuth = async (req, res, next) => {
  const token = req.headers.authorization?.split(' ')[1];
  if (!token) return res.status(401).json({ error: 'No token' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch {
    res.status(401).json({ error: 'Invalid token' });
  }
};

// Apply to specific routes only
app.get('/profile', requireAuth, (req, res) => {
  res.json(req.user); // req.user set by middleware
});

// Third-party middleware
const helmet = require('helmet');
const cors   = require('cors');
app.use(helmet()); // sets secure HTTP headers
app.use(cors({ origin: 'https://myapp.com' })); // CORS policy
```

CHAPTER 7 — FILE HANDLING WITH MULTER

File Uploads

```
const multer  = require('multer');
const path    = require('path');

// Disk storage — saves to filesystem
const storage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, 'uploads/'),
  filename:    (req, file, cb) => {
    const unique = Date.now() + '-' + Math.random().toString(36).slice(2);
    cb(null, unique + path.extname(file.originalname));
  },
});

// Validation filter
const fileFilter = (req, file, cb) => {
  const allowed = ['image/jpeg','image/png','image/webp'];
  if (allowed.includes(file.mimetype)) cb(null, true);
  else cb(new Error('Only images allowed'), false);
};

const upload = multer({ storage, fileFilter, limits: { fileSize: 5*1024*1024 } });

// Route — single file upload
app.post('/upload/avatar', upload.single('avatar'), (req, res) => {
  const file = req.file;  // { filename, path, size, mimetype }
  if (!file) return res.status(400).json({ error: 'No file uploaded' });
  res.json({ url: `/uploads/${file.filename}` });
});

// Memory storage + Cloudinary
const cloudinary  = require('cloudinary').v2;
const memStorage  = multer.memoryStorage();
const uploadMem   = multer({ storage: memStorage });

app.post('/upload/cloud', uploadMem.single('image'), async (req, res) => {
  const b64  = Buffer.from(req.file.buffer).toString('base64');
  const uri  = `data:${req.file.mimetype};base64,${b64}`;
  const result = await cloudinary.uploader.upload(uri, { folder: 'avatars' });
  res.json({ url: result.secure_url });
});
```

CHAPTER 8 — MONGODB & MONGOOSE

Document Database Design

Users Collection

```
{ _id: ObjectId('...'),
  name: 'Alice Johnson',
  email: 'alice@dev.io',
  age: 27,
  role: 'admin',
  createdAt: ISODate(...),
  address: {
    city: 'Mumbai',
    pin: '400001'
  },
  tags: ['react','node']
}
```

Mongoose Schema

```
const userSchema = new Schema({
  name:  { type: String,
           required: true },
  email: { type: String,
           unique: true },
  age:   { type: Number,
           min: 0 },
  role:  { type: String,
           enum:['user','admin'],
           default: 'user' },
  createdAt: { type: Date,
               default: Date.now }
});
```

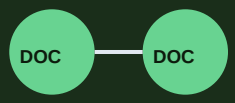
MongoDB stores JSON-like documents; Mongoose adds schema validation on top

MongoDB Database Relationships

One-to-One

User → Profile

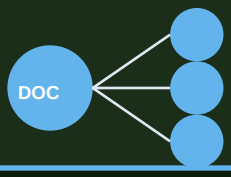
1 user has 1 profile



One-to-Many

User → Posts

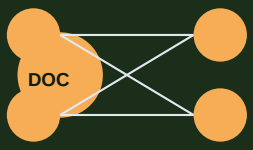
1 user has many posts



Many-to-Many

Students ↔ Courses

ref array or junction

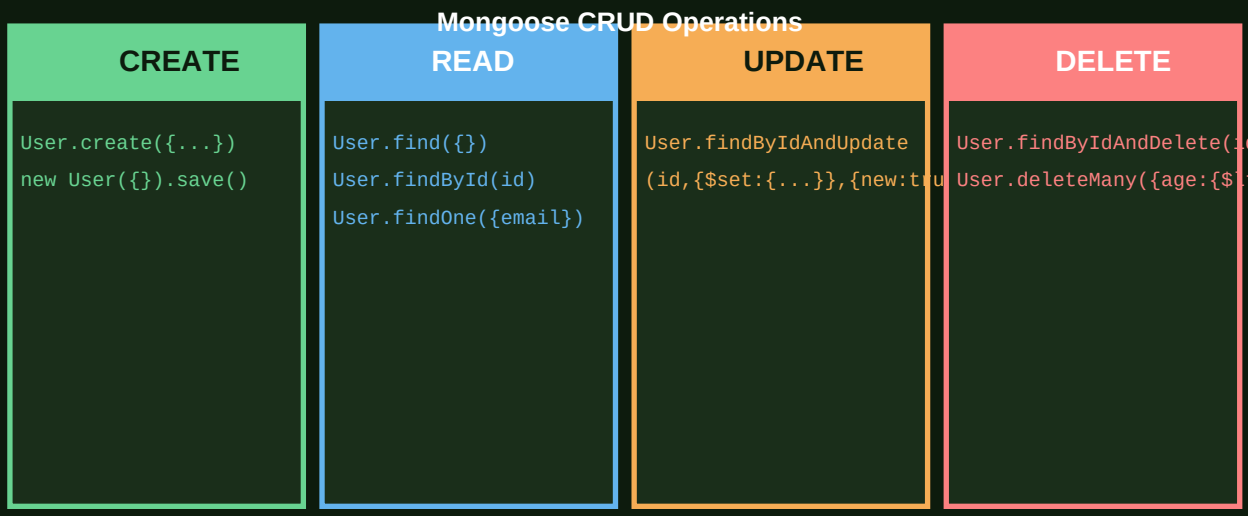


Embedded

User + Address

nest inside document

Choose embedding for tightly coupled data, references for large/shared collections



Mongoose CRUD: every operation returns a Promise — always use async/await

```
// Mongoose Model definition
const mongoose = require('mongoose');

const postSchema = new mongoose.Schema({
  title:   { type: String, required: true, trim: true },
  content: { type: String, required: true },
  author:  { type: mongoose.Schema.Types.ObjectId, ref: 'User' }, // reference
  tags:    [String],
  likes:   { type: Number, default: 0 },
  published: { type: Boolean, default: false },
}, { timestamps: true });

// Virtual field (not stored in DB)
postSchema.virtual('summary').get(function() {
  return this.content.slice(0, 100) + '...';
});

// Instance method
postSchema.methods.publish = function() {
  this.published = true; return this.save();
};

// Static method
postSchema.statics.findByTag = function(tag) {
  return this.find({ tags: tag });
};

const Post = mongoose.model('Post', postSchema);

// Populate — resolve references (like SQL JOIN)
const posts = await Post.find().populate('author', 'name email');
// author field now contains the full User object
```

CHAPTER 9 — REST API DEVELOPMENT

Building Production REST APIs

```
// Versioned API router
// routes/v1/user.routes.js
const router = require('express').Router();

router.get('/',         userController.getAll);    // GET  /api/v1/users
router.get('/:id',      userController.getById);   // GET  /api/v1/users/:id
router.post('/',        userController.create);    // POST /api/v1/users
router.patch('/:id',    userController.update);    // PATCH /api/v1/users/:id
router.delete('/:id',   userController.delete);    // DELETE /api/v1/users/:id

// Mount with version prefix
// app.use('/api/v1/users', userRoutes);

// Controller with proper error handling
const getById = async (req, res, next) => {
  try {
    const user = await User.findById(req.params.id);
    if (!user) return res.status(404).json({ success: false, message: 'Not found' });
    res.json({ success: true, data: user });
  } catch (err) {
    next(err); // pass to error handler
  }
};

// Input validation with Zod
const { z } = require('zod');
const createUserSchema = z.object({
  name:     z.string().min(2).max(50),
  email:    z.string().email(),
  password: z.string().min(8),
});

const validateBody = (schema) => (req, res, next) => {
  const result = schema.safeParse(req.body);
  if (!result.success) return res.status(422).json({ errors: result.error.issues });
  req.body = result.data; next();
};

router.post('/', validateBody(createUserSchema), userController.create);
```

CHAPTER 10 — DATABASE OPTIMIZATION

MongoDB Indexing & Aggregation

Indexing — Why It Matters

Without index: MongoDB scans EVERY document (COLLSCAN) — O(n) performance.

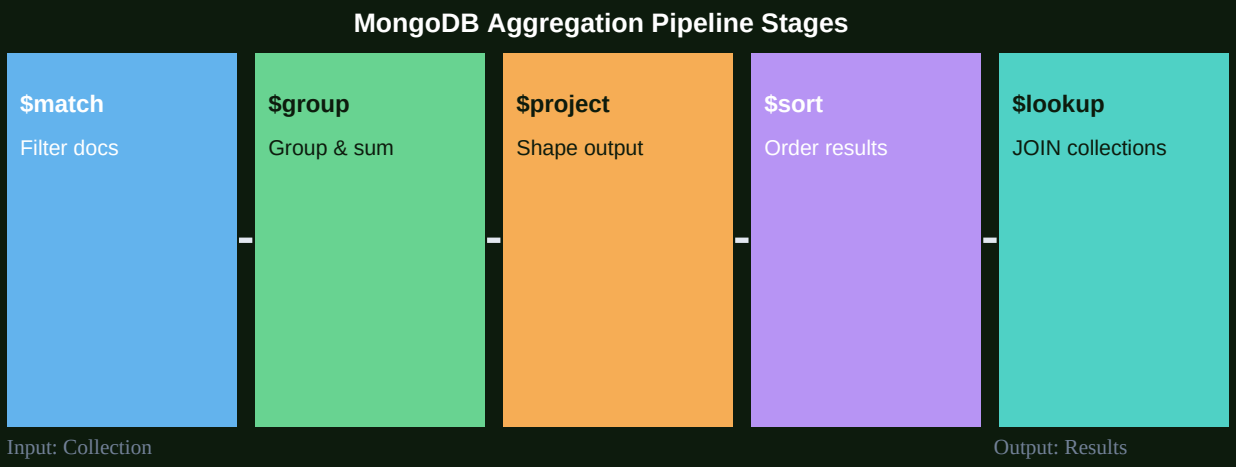
With index: MongoDB jumps directly to matching documents (IXSCAN) — O(log n).

Rule of thumb: Index fields you query/sort on frequently.

Cost: Indexes slow down writes (index must be updated) and use disk space.

Use explain('executionStats') to verify an index is being used.

```
// Creating indexes
userSchema.index({ email: 1 }, { unique: true });      // single field
postSchema.index({ author: 1, createdAt: -1 });        // compound (1=asc, -1=desc)
productSchema.index({ name: 'text', description: 'text' }); // text search

// Query with explain to check index usage
await User.find({ email: 'alice@dev.io' }).explain('executionStats');
// Look for: 'IXSCAN' (good) vs 'COLLSCAN' (bad)
```



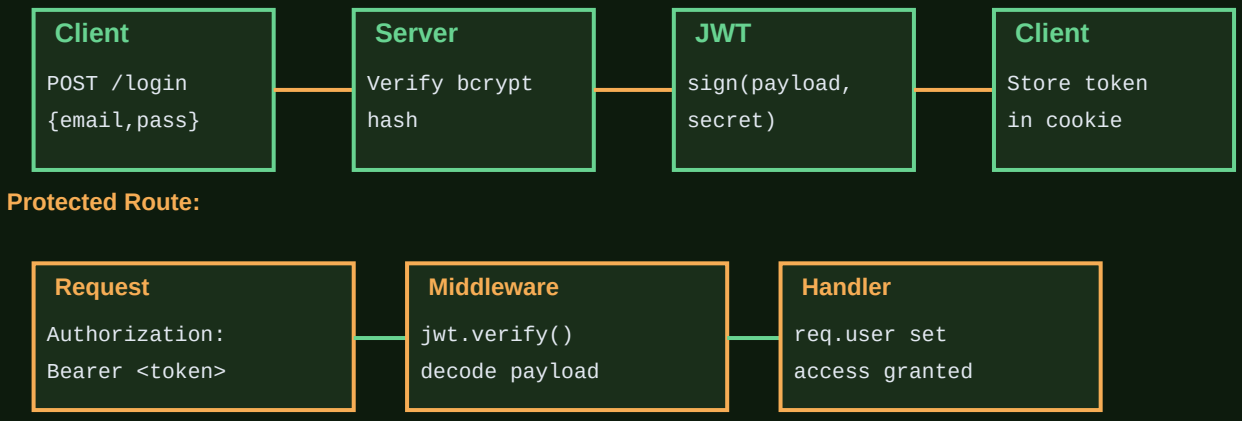
Aggregation pipeline: chain stages to transform and analyze data

```
// Aggregation example: sales report by category
const report = await Order.aggregate([
  { $match: { status: 'completed', createdAt: { $gte: lastMonth } } },
  { $group: {
    _id: '$category',
    totalRevenue: { $sum: '$amount' },
    orderCount:   { $sum: 1 },
    avgOrder:     { $avg: '$amount' }
  }},
  { $sort: { totalRevenue: -1 } },
  { $limit: 10 },
  { $project: {
    category: '$_id',
    totalRevenue: 1,
    orderCount: 1,
    avgOrder: { $round: ['$avgOrder', 2] }
  }}
]);
```

CHAPTER 11 — AUTHENTICATION & AUTHORIZATION

JWT & bcrypt Authentication

JWT Authentication Flow



JWT flow: login → sign token → client stores → send on every protected request

```
const bcrypt = require('bcryptjs');
const jwt    = require('jsonwebtoken');

// REGISTER — hash password before saving
const register = async (req, res) => {
  const { name, email, password } = req.body;
  const existing = await User.findOne({ email });
  if (existing) return res.status(409).json({ error: 'Email taken' });

  const salt   = await bcrypt.genSalt(12);  // higher = slower = safer
  const hashed = await bcrypt.hash(password, salt);
  const user   = await User.create({ name, email, password: hashed });

  const token  = jwt.sign(
    { id: user._id, role: user.role },  // payload
    process.env.JWT_SECRET,              // secret
    { expiresIn: '7d' }                  // expiry
  );

  res.status(201).json({ token });
};

// LOGIN — verify password
const login = async (req, res) => {
  const { email, password } = req.body;
  const user = await User.findOne({ email }).select('+password');
  if (!user) return res.status(401).json({ error: 'Invalid credentials' });

  const match = await bcrypt.compare(password, user.password);
  if (!match) return res.status(401).json({ error: 'Invalid credentials' });

  const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '7d' });
  res.json({ token });
};

// RBAC — Role-Based Access Control
const requireRole = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role))
    return res.status(403).json({ error: 'Forbidden' });
  next();
};

router.delete('/users/:id', requireAuth, requireRole('admin'), deleteUser);
```

CHAPTER 12 — WEBSOCKETS & SOCKET.IO

Real-Time Communication

HTTP Polling vs WebSocket

HTTP Polling

C→S: GET /messages

S→C: [] (empty)

C→S: GET /messages

S→C: [] (empty)

C→S: GET /messages

S→C: [new msg!]

Wastes bandwidth!

WebSocket

C→S: Upgrade request

S→C: 101 Switching

— Connection open —

S→C: msg instantly

C→S: msg instantly

← Persistent connection →

Real-time! Zero waste.

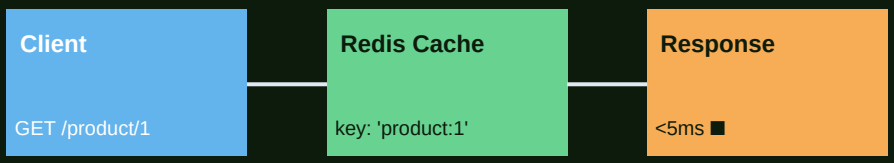
WebSocket vs polling: persistent bidirectional connection vs repeated HTTP requests

```
const { Server } = require('socket.io');
const io = new Server(httpServer, {
  cors: { origin: 'http://localhost:3000' }
});

// Auth middleware for Socket.io
io.use((socket, next) => {
  const token = socket.handshake.auth.token;
  try { socket.user = jwt.verify(token, process.env.JWT_SECRET); next(); }
  catch { next(new Error('Unauthorized')); }
});

io.on('connection', (socket) => {
  console.log(`User connected: ${socket.user.id}`);

  // Join a room (e.g. chat room)
  socket.on('join-room', (roomId) => {
    socket.join(roomId);
    socket.to(roomId).emit('user-joined', { user: socket.user });
  });

  // Handle chat message
  socket.on('message', async ({ roomId, content }) => {
    const msg = await Message.create({ user: socket.user.id, content, room: roomId });
    io.to(roomId).emit('new-message', msg); // broadcast to ALL in room
  });

  socket.on('disconnect', () => {
    console.log(`User disconnected: ${socket.user.id}`);
  });
});

// Client-side
const socket = io('http://localhost:3000', { auth: { token } });
socket.emit('join-room', 'general');
socket.on('new-message', (msg) => appendMessage(msg));
```

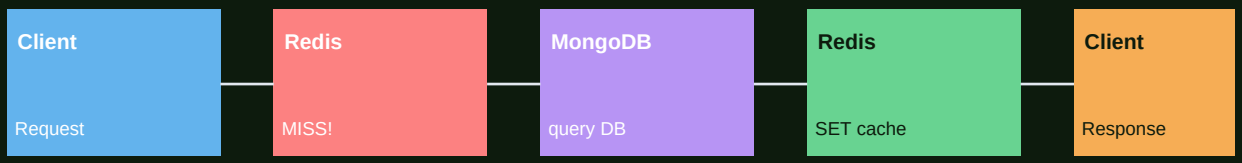
CHAPTER 13 — REDIS CACHING

Caching for Performance

Cache HIT



Cache MISS

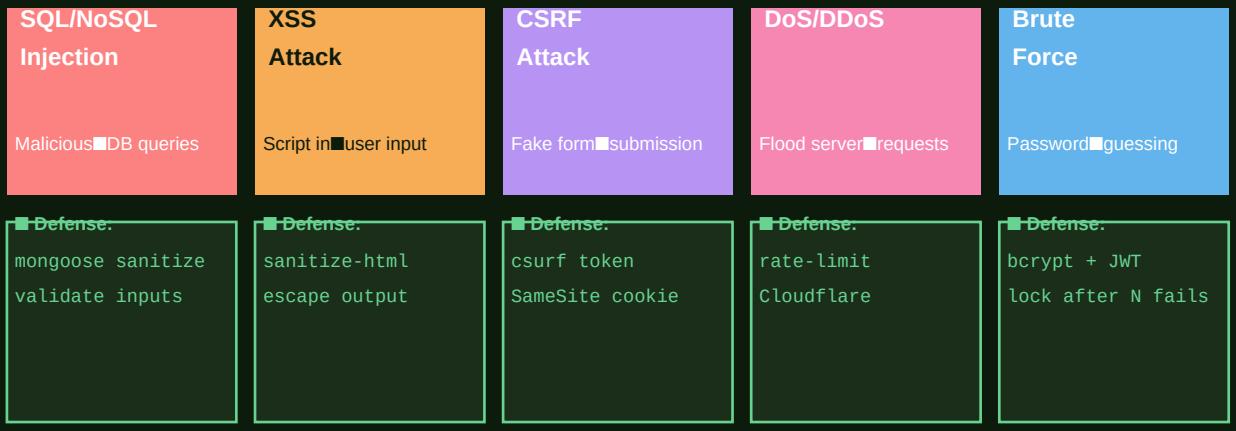


Cache hit: serve from Redis in <5ms. Cache miss: query DB, store in Redis, serve

```
const { createClient } = require('redis');
const redis = createClient({ url: process.env.REDIS_URL });
await redis.connect();

// Cache middleware factory
const cache = (ttl = 60) => async (req, res, next) => {
  const key = `cache:${req.originalUrl}`;
  const cached = await redis.get(key);
  if (cached) return res.json(JSON.parse(cached));  // HIT!

  // Override res.json to store result
  const originalJson = res.json.bind(res);
  res.json = (data) => {
    redis.setEx(key, ttl, JSON.stringify(data));    // store for ttl seconds
    return originalJson(data);
  };
  next(); // MISS — proceed to handler
};

// Usage
router.get('/products', cache(300), productController.getAll); // 5 min cache

// Manual invalidation
const invalidateCache = async (pattern) => {
  const keys = await redis.keys(pattern);
  if (keys.length) await redis.del(keys);
};

// After product update — clear related caches
await invalidateCache('cache:/products*');

// Pub/Sub for real-time notifications
const subscriber = redis.duplicate();
await subscriber.connect();
await subscriber.subscribe('notifications', (message) => {
  io.emit('notification', JSON.parse(message));
});

// Publish from another service
await redis.publish('notifications', JSON.stringify({ type: 'order', id: 123 }));
```

CHAPTER 14 — SECURITY & ERROR HANDLING

Production Security



Know your threats, implement defenses at every layer of the stack

```
const rateLimit    = require('express-rate-limit');
const mongoSanitize = require('express-mongo-sanitize');
const xss           = require('xss-clean');

// Rate limiting — prevent brute force & DoS
const limiter = rateLimit({
  windowMs: 15 * 60 * 1000,  // 15 minutes
  max: 100,                   // max 100 requests per window
  message: { error: 'Too many requests, try again in 15 minutes' },
});
app.use('/api/', limiter);

// Auth route stricter limit
const authLimiter = rateLimit({ windowMs: 60*60*1000, max: 10 });
app.use('/api/v1/auth', authLimiter);

// Sanitize against NoSQL injection: { email: { $gt: '' } }
app.use(mongoSanitize());

// Sanitize against XSS: <script>alert('xss')</script>
app.use(xss());

// Global error handler (must have 4 params!)
app.use((err, req, res, next) => {
  const statusCode = err.statusCode || 500;
  const message    = err.message    || 'Internal server error';

  if (process.env.NODE_ENV === 'development') {
    res.status(statusCode).json({ success: false, message, stack: err.stack });
  } else {
    res.status(statusCode).json({ success: false, message });
  }
});

// Custom API Error class
class ApiError extends Error {
  constructor(statusCode, message) {
    super(message);
    this.statusCode = statusCode;
  }
};

// Usage: throw new ApiError(404, 'User not found');
```

CHAPTER 15 — PRODUCTION & TESTING

Production-Ready Setup

```
// .env — environment variables (NEVER commit this!)
NODE_ENV=production
PORT=3000
MONGO_URI=mongodb+srv://user:pass@cluster.mongodb.net/mydb
JWT_SECRET=super-long-random-secret-here
REDIS_URL=redis://localhost:6379
CLOUDINARY_URL=cloudinary://...

// PM2 — process manager for production
# npm install -g pm2
# pm2 start src/index.js --name 'my-api' --instances max
# pm2 restart my-api   # zero-downtime restart
# pm2 logs my-api      # view logs
# pm2 monit             # dashboard

// ecosystem.config.js (PM2 config)
module.exports = {
  apps: [{
    name: 'my-api',
    script: 'src/index.js',
    instances: 'max',
    exec_mode: 'cluster',
    env_production: {
      NODE_ENV: 'production',
      PORT: 3000
    }
  }]
};
```

Unit Testing with Jest

```
// user.test.js
const request = require('supertest');
const app     = require('../src/app');
const mongoose = require('mongoose');

beforeAll(async () => {
  await mongoose.connect(process.env.MONGO_TEST_URI);
});
afterAll(async () => {
  await mongoose.connection.dropDatabase();
  await mongoose.connection.close();
});

describe('POST /api/v1/auth/register', () => {
  it('should create a new user', async () => {
    const res = await request(app).post('/api/v1/auth/register').send({
      name: 'Alice', email: 'alice@test.com', password: 'Password123'
    });

    expect(res.status).toBe(201);
    expect(res.body.token).toBeDefined();
  });

  it('should reject duplicate email', async () => {
    await request(app).post('/api/v1/auth/register').send({
      name: 'Bob', email: 'alice@test.com', password: 'Password123'
    });

    const res = await request(app).post('/api/v1/auth/register').send({
      name: 'Bob2', email: 'alice@test.com', password: 'Password123'
    });

    expect(res.status).toBe(409);
  });
});
```

EPISODE 3 COMPLETE — WHAT YOU'VE LEARNED

✓ **Node.js & Event Loop**	V8 engine, libuv, non-blocking I/O, npm
✓ **HTTP Server & Express**	Routing, query params, static files, Nodemon
✓ **MVC Architecture**	Models, controllers, routes, folder structure
✓ **Middleware**	Built-in, custom, auth, Helmet, CORS, error handling
✓ **MongoDB & Mongoose**	Schemas, CRUD, relations, populate, virtuals
✓ **REST API Design**	Versioning, HTTP methods, status codes, Postman
✓ **Input Validation**	Zod schemas, sanitization, error responses

✓ **Aggregation & Indexing**	Pipeline stages, compound indexes, explain()
✓ **JWT Authentication**	bcrypt hashing, token signing, protected routes, RBAC
✓ **WebSockets**	Socket.io, rooms, real-time events, auth middleware
✓ **Redis Caching**	Cache middleware, TTL, invalidation, Pub/Sub
✓ **Security**	Rate limiting, XSS, NoSQL injection, global error handler
✓ **Production**	PM2, .env, cluster mode, logging
✓ **Testing**	Jest, Supertest, unit & integration tests

What's Next — Episode 4 & 5

Episode 4: Generative AI Engineering

→ LLM APIs, prompt engineering, structured outputs, function calling

→ Embeddings, vector DBs, RAG pipelines, AI agents, LangChain

Episode 5: Production Systems, DevOps & Deployment

→ Docker, AWS EC2, Load Balancers, CI/CD with GitHub Actions

→ CloudFront CDN, security groups, scaling strategies